CSE 4105 · SYSTEM PROGRAMMING · SHORT COURSE

Class 1

What System Programming Is

from hello.c to a running program, and the machine underneath

CLASS

1 / 10

17 slides, taken from the full course:
Lecture 1

Md Mahbub E Noor
Assistant Professor, Department of CSE
University of Barishal

Application software vs system software

Application software

Solves **the user's** problem. The user knows it exists and talks to it directly.

Examples: Web browser, MS Word, a game, the spam detector you built in Python.

System software

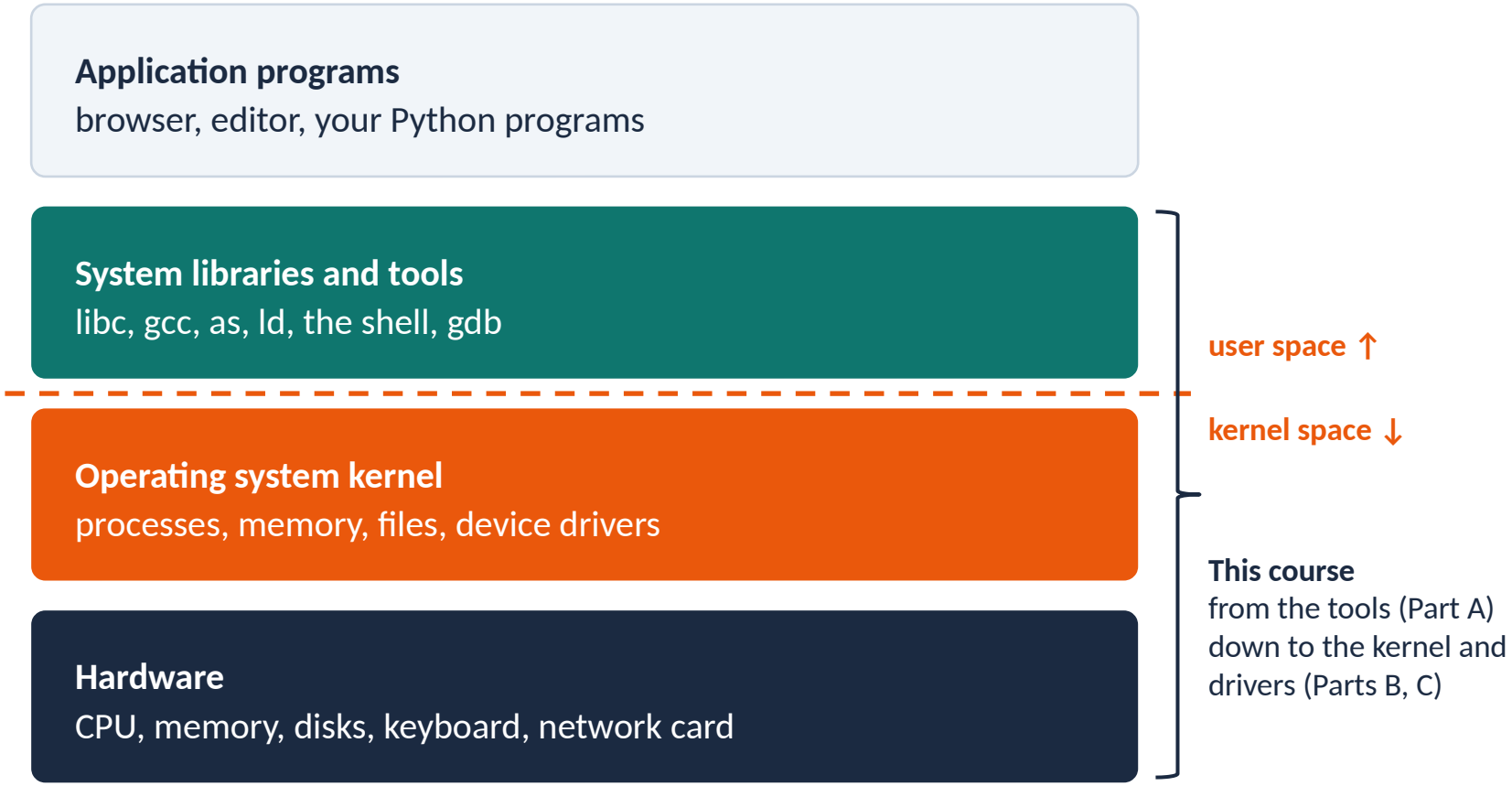
Supports **the operation of the computer itself**. Other programs run on top of it. Most users never notice it.

Examples: Compiler, assembler, linker, loader, operating system kernel, device drivers.

Key idea. System software depends on the machine it runs on. An assembler must know the CPU's **instruction formats and addressing modes**; a loader must know how **memory addresses** work. Some features do not depend on the machine (for example, how an assembler stores its symbol table). We will label every feature as **machine-dependent** or **machine-independent**.

Book: Beck, System Software, §1.1–1.2

Where system software lives: the layered view



Each layer only talks to the layer below it. Your program never touches the disk itself. It calls a library function, the library asks the kernel, and the kernel uses a driver to talk to the disk.

The boundary between **user space** and **kernel space** is guarded by the CPU itself. Crossing it needs a special instruction, a **system call** (Lecture 9).

Book: Bryant & O'Hallaron, CS:APP Fig. 1.10; Love, LKD Fig. 1.1

From hello.c to a running process: five steps

hello.c

```c
#include <stdio.h>

int main(void)
{
  printf("Hello, ...\n");
  return 0;
}
```

gcc is not one program. It is a driver that runs four other programs in turn. The kernel then runs the fifth step when you type `./hello`.

1	**Preprocess** `cpp`	`hello.c → hello.i`	Pastes in the #include files and expands macros. Our 7 lines become 820.	L4
2	**Compile** `cc1`	`hello.i → hello.s`	Translates C into assembly language for this CPU (x86-64).	—
3	**Assemble** `as`	`hello.s → hello.o`	Turns each mnemonic into machine-code bytes. Leaves holes for addresses it does not know yet.	L3
4	**Link** `ld`	`hello.o + libc → hello`	Fills the holes, attaches the C library, writes an executable ELF file.	L5, L6
5	**Load** `kernel`	`hello → process`	Creates a process, maps the file into memory, jumps to its first instruction.	L5, L8

Book: Bryant & O'Hallaron, CS:APP Fig. 1.3 (the compilation system)

Demo: stop gcc after every step

```
bash — ~/lab1
$ gcc -E hello.c -o hello.i  # stop after preprocessing
$ gcc -S hello.c -o hello.s  # stop after compiling
$ gcc -c hello.s -o hello.o  # stop after assembling
$ gcc hello.o -o hello       # link
$ ./hello
Hello, System Programming!
$ wc -l hello.c hello.i hello.s
$ ls -l hello*  ;  file hello*
```

File	Lines	Bytes	What the file command says
`hello.c`	7	97	C source, ASCII text
`hello.i`	820	21,329	C source, ASCII text
`hello.s`	45	681	assembler source, ASCII text
`hello.o`	—	1,512	ELF 64-bit LSB relocatable, x86-64
`hello`	—	15,960	ELF 64-bit LSB pie executable, x86-64, dynamically linked

Look closely. hello.i is **120 times bigger** than hello.c: that is stdio.h pasted in. The last two files are both **ELF** files, but only `hello` can run. `hello.o` is "relocatable": its addresses are not final yet.

Question: what exactly is missing from hello.o? (Next two slides, and Lectures 5–6.)

Inside hello.s: your first look at assembly

hello.s (main only, directives removed)

```
main:
    endbr64
    pushq   %rbp
    movq    %rsp, %rbp
    leaq    .LC0(%rip), %rax
    movq    %rax, %rdi
    call    puts@PLT
    movl    $0, %eax
    popq    %rbp
    ret
```

`pushq / movq`

Set up a **stack frame** for main, so it has its own space on the stack.

`leaq .LC0(%rip), %rax`

Put the **address of the string** "Hello, ..." into register rax.

`movq %rax, %rdi`

Copy it to **rdi**: on x86-64 Linux the first argument always travels in rdi.

`call puts@PLT`

Jump to the library function **puts**, through the PLT (Lecture 5.2).

`movl $0, %eax`

The return value goes in **eax**: this is `return 0;`

Puzzle. We wrote **printf**, but the compiler called **puts**. Why is that allowed? (Hint: look at what our string ends with, and whether it has any % signs.)

Book: CS:APP §3.2–3.4 (x86-64 registers and calling convention)

Inside hello.o: machine code with holes in it

```
objdump -d hello.o

0000000000000000 <main>:
   0:  f3 0f 1e fa           endbr64
   4:  55                    push   %rbp
   5:  48 89 e5              mov    %rsp,%rbp
   8:  48 8d 05 00 00 00 00  lea    0x0(%rip),%rax
   f:  48 89 c7              mov    %rax,%rdi
  12:  e8 00 00 00 00        call   17 <main+0x17>
  17:  b8 00 00 00 00        mov    $0x0,%eax
  1c:  5d                    pop    %rbp
  1d:  c3                    ret
```

```
nm hello.o

0000000000000000 T main
                 U puts
```

Read a line

Left: the **offset** inside main. Middle: the **machine-code bytes**. Right: the same instruction in assembly.

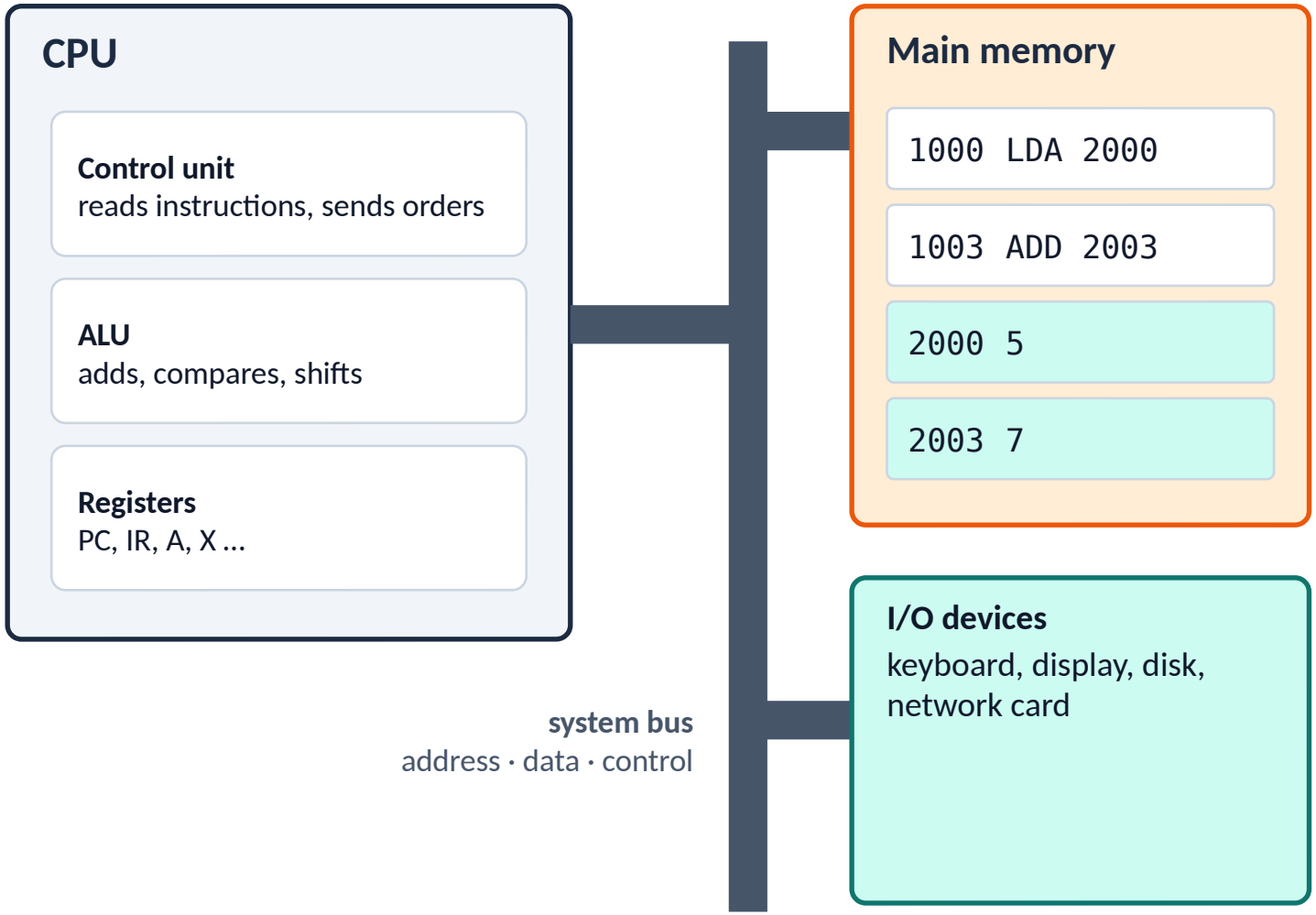
Find the holes

The orange lines end in **00 00 00 00**. The assembler did not know the address of the string or of puts, so it wrote zeros and left a note, a **relocation entry**, for the linker.

Read nm

T main: main is defined here, in the code ("text") section. **U puts**: puts is used but **undefined** here. The linker must find it in libc.

Inside a computer: the stored-program machine



Stored-program idea. Instructions and data sit in the **same memory**, both as plain numbers. Look at memory 1000: the number there is an instruction only because the CPU **fetches** it as one.

The CPU never touches a device directly by itself. Everything travels over the **bus**: the CPU puts an address on it, and memory or a device answers.

This design is called the **von Neumann architecture**. Nearly every computer today, from a phone to a server, follows it.

Book: CS:APP Fig. 1.4; Beck §1.3 (SIC machine architecture)

Registers: the CPU's own tiny, fast memory

A register holds one word inside the CPU itself. Every CPU gives its registers the same jobs; only the names change. We use **SIC** for the assembler theory on paper (Lectures 2–5) and **x86-64** for the real labs.

SIC and SIC/XE (Beck)

Name	No.	Job
A	0	Accumulator: arithmetic results
X	1	Index: stepping through arrays
L	2	Linkage: return address of a JSUB call
B	3	Base: base-relative addressing (XE)
S, T	4, 5	General working registers (XE)
F	6	Floating point, 48 bits (XE)
PC	8	Program counter: next instruction
SW	9	Status word: condition code

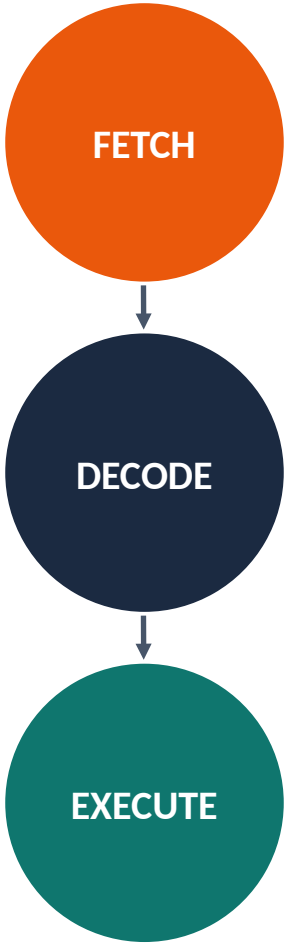
x86-64 (your lab PC)

Name	Job
rax	Return value of a function
rdi, rsi, rdx	Arguments 1, 2, 3
rcx, r8, r9	Arguments 4, 5, 6
rsp	Stack pointer: top of the stack
rbp	Base (frame) pointer
rip	Instruction pointer: same job as PC
rflags	Flags: same job as SW
rbx, r10–r15	General purpose

Orange rows: the program counter and the status register. Both machines have them, under different names.

Book: Beck §1.3.1–1.3.2; CS:APP §3.4

The fetch–decode–execute cycle




```
IR ← memory[PC]
PC ← PC + 3
```

Copy the instruction that PC points to into IR. Move PC on to the next one.

```
look at the opcode
```

The control unit reads the opcode in IR and decides which operation and which operand.

```
do the work
```

The ALU computes, or memory is read or written. Then the cycle starts again.

↻ **then back to FETCH for the next instruction**

a tiny SIC program

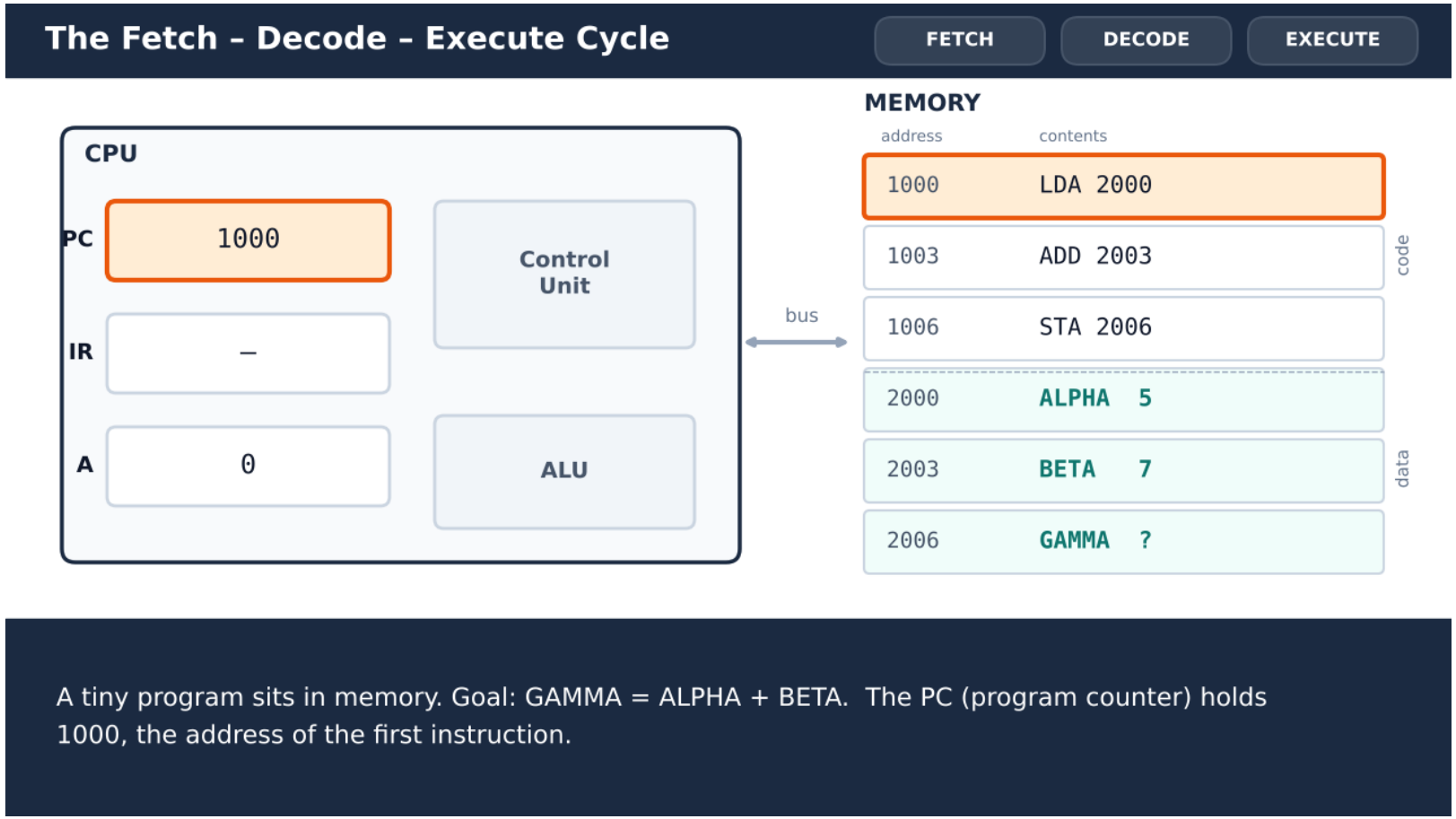
```
addr  instruction   meaning
1000  LDA  ALPHA    A ← ALPHA
1003  ADD  BETA     A ← A + BETA
1006  STA  GAMMA    GAMMA ← A
...  data ...
2000  ALPHA  WORD 5
2003  BETA   WORD 7
2006  GAMMA  RESW 1
```

Why PC + 3? Every SIC instruction is exactly **3 bytes**: 1 byte opcode, 2 bytes for flags and address. So the next instruction always starts 3 bytes later: 1000, 1003, 1006.

(x86-64 instructions are 1 to 15 bytes, so its fetch step must decode the length first.)

Book: Beck §1.3.1 (SIC instruction format); CS:APP §1.4.1

Watch the cycle run



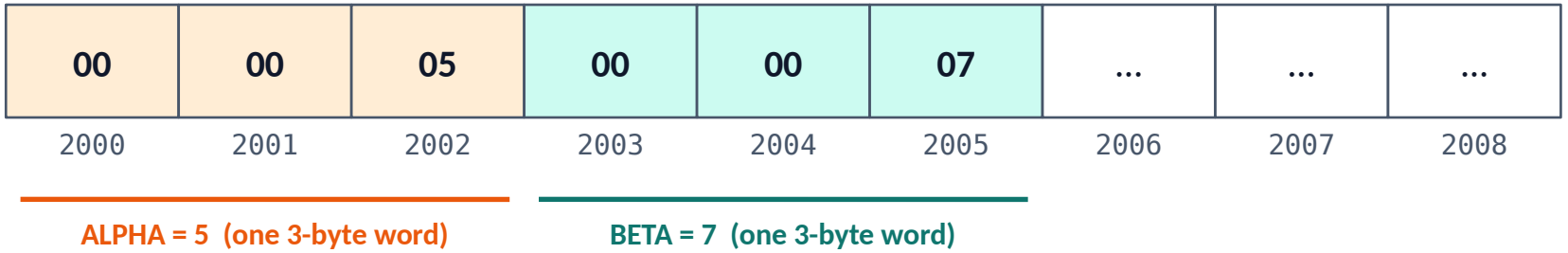
Click the video to play or pause. 55 seconds, no sound.

Watch for

1. The **phase light** at the top right: FETCH, DECODE, EXECUTE.
2. **PC** jumps by 3 at every fetch.
3. **IR** changes only during FETCH.
4. Only EXECUTE touches the **data** area of memory.
5. After STA, GAMMA becomes 12, but **A keeps 12** too.

Memory: bytes, words and addresses

Memory is one long row of **bytes**. Each byte has a number, its **address**. A **word** is the group of bytes the CPU moves in one go.



SIC word = **3 bytes** (24 bits); memory = **32,768 bytes** (2^{15}), so a 15-bit address is enough.

SIC/XE same 3-byte word; memory up to **1 MB** (2^{20}), so addresses need 20 bits (format 4).

x86-64 every byte has its own address; **pointers are 8 bytes**, as the program on the right shows.

sizes.c → ./sizes

```
# sizeof each C type
char   : 1 byte
short  : 2 bytes
int    : 4 bytes
long   : 8 bytes
pointer: 8 bytes

# 8-byte pointer =
# 64-bit addresses
```

Book: Beck §1.3.1–1.3.2; CS:APP §2.1

Byte order: which byte comes first?

A 4-byte integer `0x12345678` needs four memory cells. CPUs disagree about the order in which to store them.

Little-endian · x86-64, most ARM phones

78	56	34	12
addr+0	addr+1	addr+2	addr+3

lowest address holds the **least** significant byte

Big-endian · network byte order, many old CPUs

12	34	56	78
addr+0	addr+1	addr+2	addr+3

lowest address holds the **most** significant byte

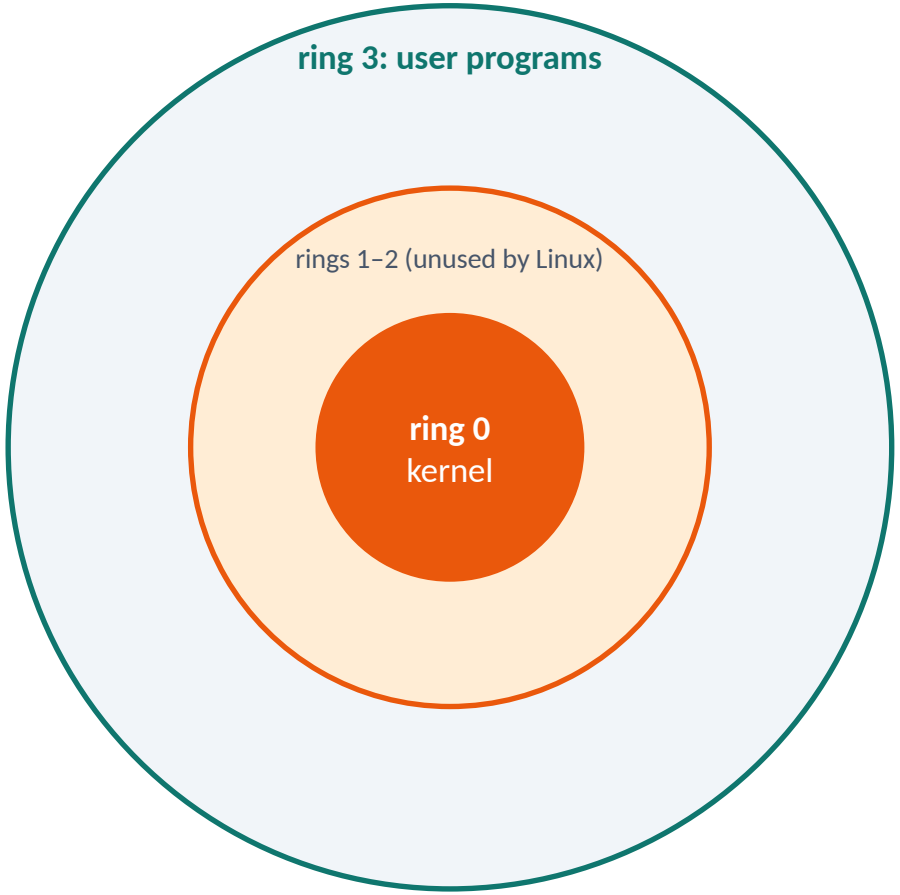
```
endian.c → ./endian
Value of x      : 0x12345678
Size of x       : 4 bytes
Address ...988 : 78
Address ...989 : 56
Address ...98a : 34
Address ...98b : 12
```

Predict first: our lab PC is x86-64. Which byte will the program print first? Then click to see the real output.

Why care? You will read raw bytes in `objdump`, `hexdump` and ELF files (Lecture 6). Read them backwards on x86-64.

Book: CS:APP §2.1.3 (addressing and byte ordering)

Two modes of the CPU: user mode and kernel mode



	User mode	Kernel mode
Who runs here	your programs, libc, the shell	the Linux kernel, drivers
Privileged instructions (hlt, cli, load page table)	not allowed	allowed
Memory it can reach	only its own	all of it
Talk to devices directly	no, must ask the kernel	yes
If it crashes	only that program dies	the whole machine can stop

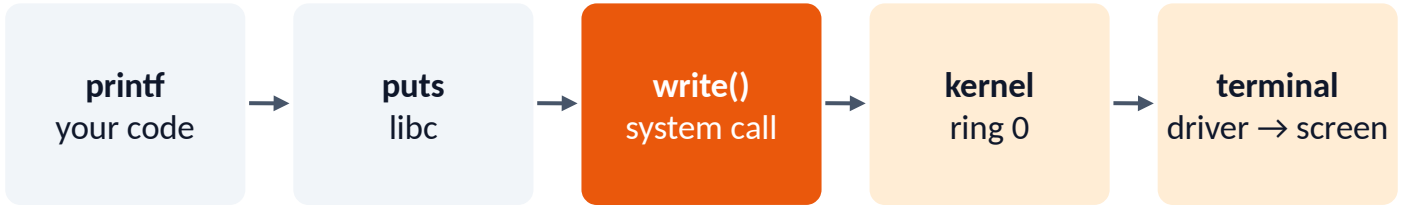
The mode is a bit inside the CPU, not a setting in software. The CPU checks it **before every privileged instruction**. x86 has four rings; Linux uses only ring 0 and ring 3.

Book: Bovet & Cesati, UTLK Ch. 1; Love, LKD Ch. 1

System calls: the legal doorway into the kernel

```
strace ./hello   (shortened)
execve("./hello", ["./hello"], ...) = 0
brk(NULL)                          = 0x55d3b96a2000
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", ...) = 3
read(3, "\177ELF\2\1\1\3...", 832) = 832
mmap(NULL, 2174352, PROT_READ, ...) = 0x7f0646000000
      ... 25 more calls to set up the process ...
write(1, "Hello, System Programming!\n", 27) = 27
exit_group(0)                      = ?
```

The path of our printf, from your code to the screen:



34
system calls
(normal build)

16
system calls
(gcc -static)

A system call is a **deliberate request** from a user program for something only the kernel may do: open a file, write to the screen, create a process.

Even "hello world" crosses into the kernel **34 times**. Most of those calls load the shared C library. A static build carries libc inside the file, so it needs only 16.

Book: Love, LKD Ch. 5; Kerrisk, TLPI Ch. 3

Key ideas from Class 1

1. System software (compiler, assembler, linker, loader, kernel) sits between your program and the hardware.
2. gcc runs four steps: preprocess, compile to assembly, assemble to an object file, link to an executable.
3. The CPU repeats fetch, decode, execute; registers hold the values it is working on right now.
4. x86-64 stores the low byte of a number first (little-endian).
5. User programs run in user mode; they enter the kernel only through system calls.

Homework and practice

Homework 1 (bring it to the next class)

1. Write the four commands that stop gcc after each step, and name the file each one produces.
2. Draw how the int 0x12345678 is stored at address 1000 on x86-64, byte by byte.
3. Why can a user program not execute a privileged instruction? What happens if it tries?

Try it in the browser

Bytes in memory: watch a value become bytes

`labs/lab01.html`

On the course website: open the lab page and choose "Try it here".

Next class: Machine Language and the SIC Machine